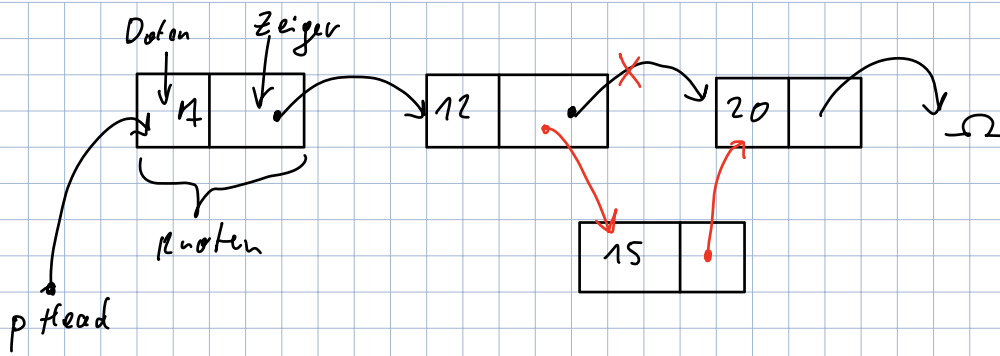
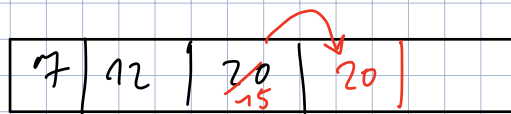




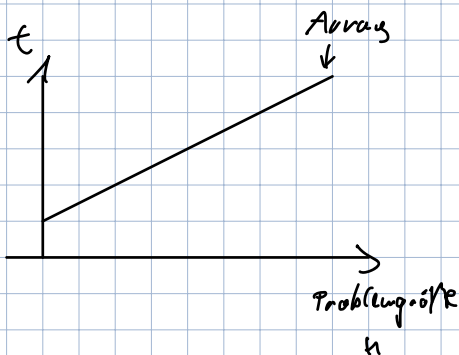
vgl. Array



Array einfügen / Löschen
im Schnitt $\frac{n}{2}$ Operationen

Asymptotische Laufzeit
Komplexität

$O(n)$



Array durchlaufen

```
for (int i = 0; i < n; i++) {
    ...
}
```

$t: f(n) = n \cdot c_1 + c_2$
 $O(n)$

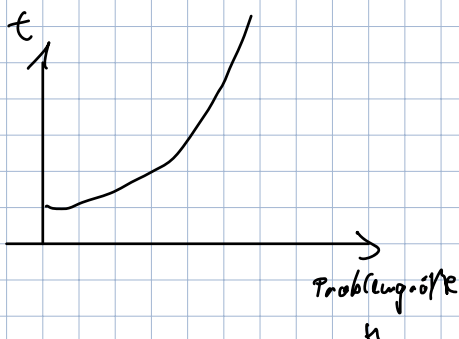
$M' = M_1 + M_2$

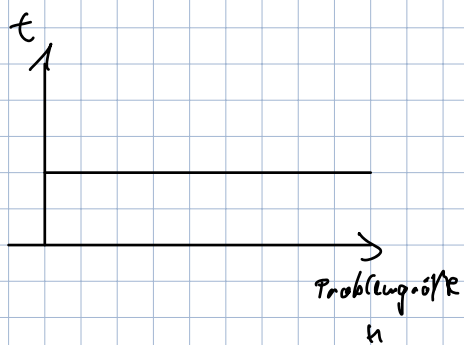
```
for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < n; j++) {
```

```
        m[i][j] = m1[i][j] + m2[i][j];
```

$f(n) = n^2 \cdot c_1 + c_2$
 $O(n^2)$





```
int f (int x) {
    return x * x;
}
```

$$f(x) = c$$

$$O(1)$$

Dynamischen Speicher reservieren

```
void* malloc (size_t size);
```

kein malloc() ohne free()!

```
void* free (void* ptr);
```

```
int n = 12
```

```
uint16_t * values = malloc (n * sizeof(uint16_t));
```

```
if (values == NULL) {
```

```
    // Fehler
```

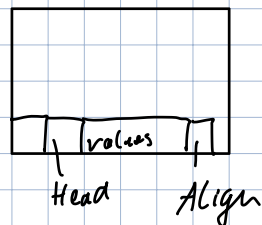
```
}
```

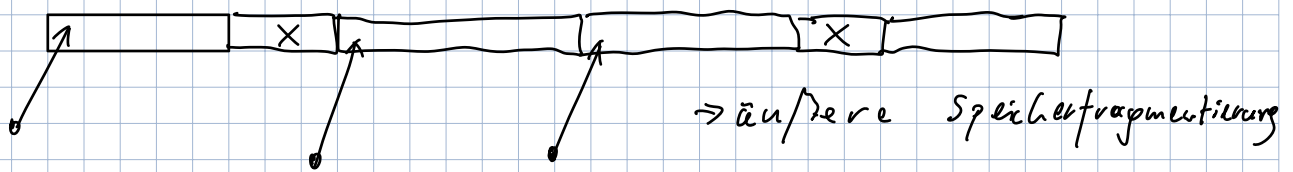
```
values[4] = 12800;
```

```
...
```

```
free (values);
```

Heap

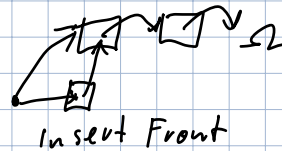




```
typedef struct _Node_ {
    uint16_t value;
    struct _Node_* pNext;
} node_t;
```

```
node_t* gpHead = NULL;
```

*node **ppHead*



*List * pList*

```
void insertFront(uint16_t value) {
```

```
    node_t* pNewNode = malloc(sizeof(node_t));
```

```
    if (pNewNode == NULL) {
```

```
        exit(-1);
```

```
    }
```

```
    pNewNode->value = value;
```

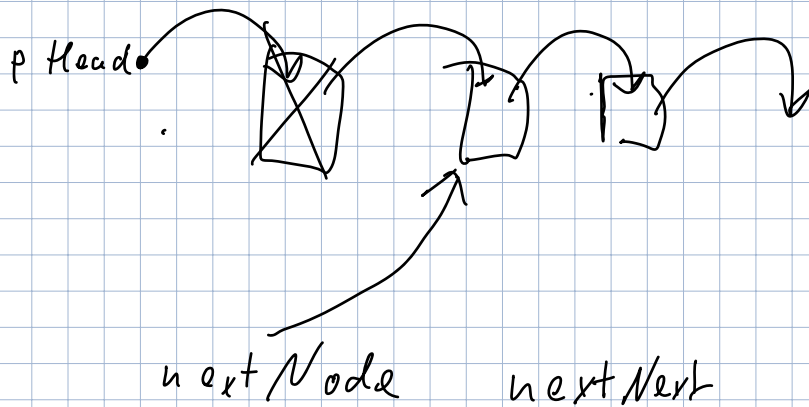
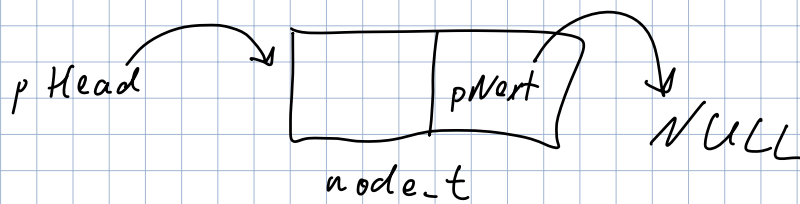
```
    pNewNode->pNext = *gpHead;  pList->pHead
```

```
    *gpHead = pNewNode;  'p
```

```
}
```

call by ref.

```
insertFront(&gpHead, 12);
```



... main ... {

```
node * pList1 = NULL;
Node * pList2 = NULL;
```

```
insertFront (&pList1, 20);
```

```
insertFront (&pList1, 12);
```

```
insertFront (&pList2, 15);
```

```
printList (pList1);
```

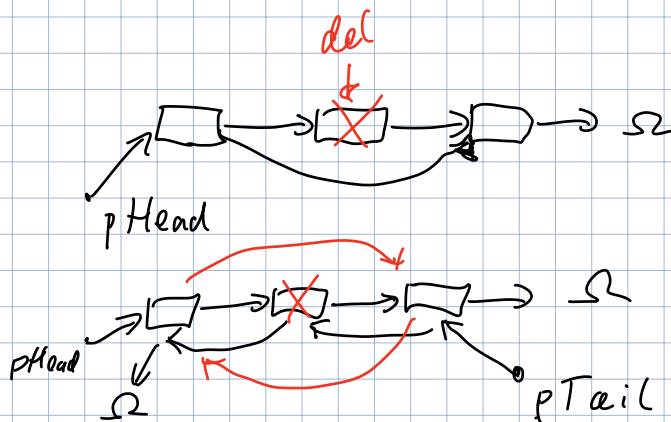
```
printList (pList2);
```

```
deleteNode (&List1, 20);
```

```

List * pList
void printList (node * pHead) {
    node * pCurrent Node = pHead;    pList → pHead;
    while (pCurrent → next != NULL) {
        printf ("%d → ", pCurrent → value);
        pCurrent = pCurrent → pNext;
    }
}

```



```

typedef struct _List_ {
    node * pHead;
    // node * pTail;
    uint32_t size;
}

```

} List;

List List 1 = {0};

List List 2 = {0};

void deleteNode (List * pList, int value){

Node * pPredecessor = NULL;

Node * pNodeToDelete = pList → pHead;

while (pNodeToDelete != NULL &&

pNodeToDelete → value != value) {

pPredecessor = pNodeToDelete;

pNodeToDelete = pNodeToDelete → pNext;

}

if (pNodeToDelete != NULL) {

if (pPredecessor != NULL) {

pPredecessor → pNext = pNodeToDelete → pNext;

} else {

pList → pHead = pNodeToDelete → pNext;

}

free(pNodeToDelete);

}

}